# Bisection Method Root Solver in MATLAB

**Numerical Methods Portfolio Project**

**Author**

**Ehsan Shakil Ahmad**

**Developed Using**

MATLAB Online

**Project Date**

June 2026

## Project Overview

This project presents the development of a MATLAB-based Bisection Method solver for finding the roots of nonlinear equations. The implementation includes automatic interval detection, convergence monitoring, error handling, and graphical visualization of results. The objective is to demonstrate the practical application of numerical root-finding algorithms and MATLAB programming for engineering analysis.

**Keywords:** Numerical Methods, MATLAB, Root Finding, Bisection Method, Engineering Computation

# TABLE OF CONTENTS

# 1 THEORY

The Bisection Method is a numerical root-finding technique that repeatedly halves an interval containing a root.

For

$$f(x) = x^3 - x - 2$$

Since

$$f(1) = -2, f(2) = 4$$

a root exists in the interval [1,2] because the function changes sign.

The midpoint is:

$$c = \frac{a + b}{2}$$

and the interval containing the sign change is selected until the desired tolerance is achieved.

# 2 NOMENCLATURE TABLE

| Symbol | Description |
|---|---|
| (a) | Lower interval bound |
| (b) | Upper interval bound |
| (c) | Midpoint |
| (f(x)) | Function being analyzed |
| tol | Convergence tolerance |
| iter | Iteration counter |

# 3 CODE IMPROVEMENT

Several improvements were incorporated into the MATLAB implementation to enhance the flexibility and robustness of the Bisection Method solver. In conventional implementations, the initial interval containing the root is manually specified by the user. In this project, an automated interval detection mechanism was developed, which scans the domain [-10,10] and identifies adjacent points where a sign change occurs. This feature eliminates the need for manual interval selection and allows the program to handle a wider range of functions.

The algorithm also includes an error-checking routine that terminates execution and displays an informative message if no sign change is detected within the specified search domain. This prevents invalid computations and improves program reliability.

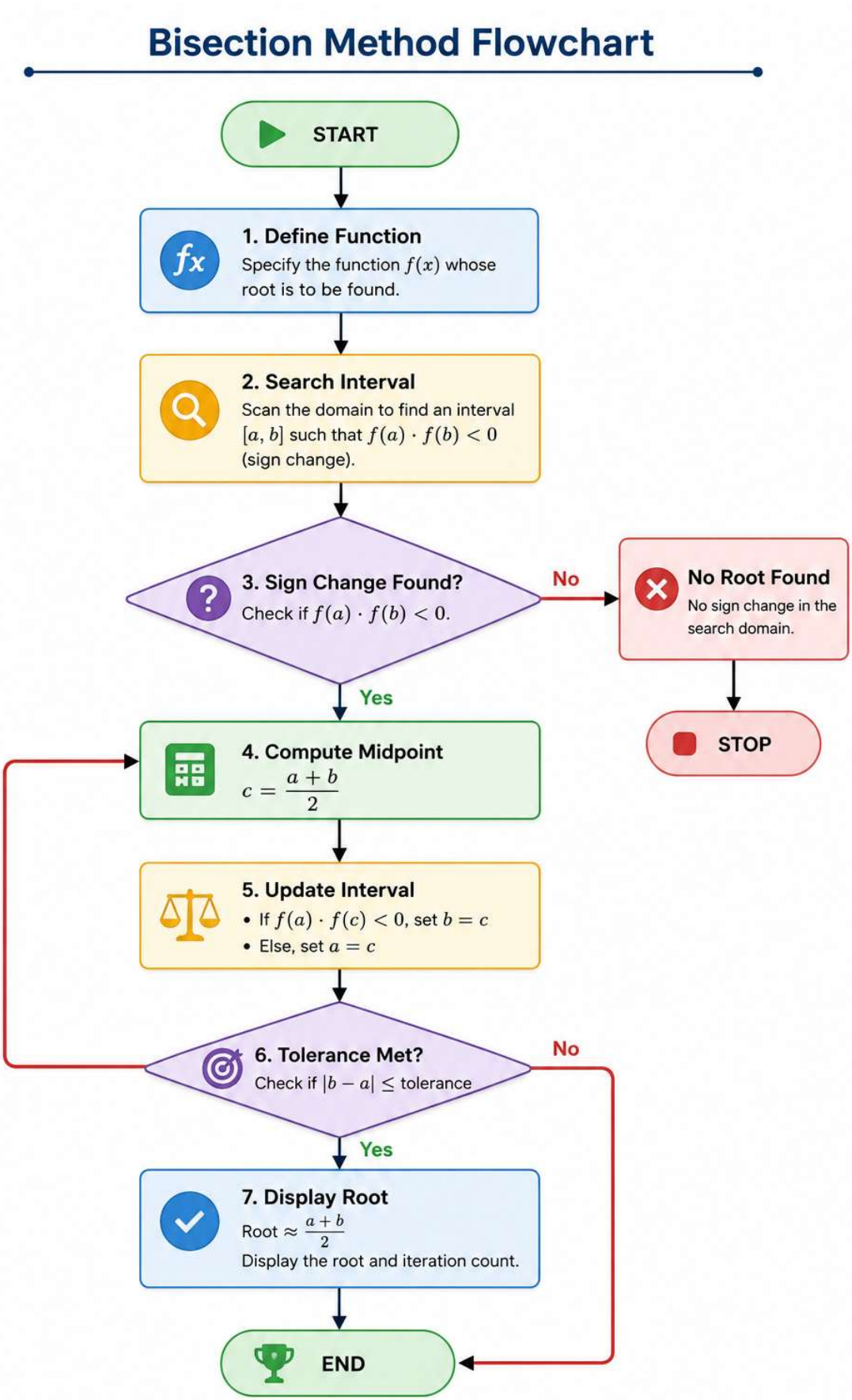
To monitor convergence, the program displays the values of the lower bound ((a)), upper bound ((b)), midpoint ((c)), and iteration number during each iteration. This provides a clear visualization of the convergence process and allows users to verify the correctness of the algorithm.

Finally, graphical visualization was incorporated by plotting the function and highlighting the computed root on the graph. This feature enables intuitive verification of the numerical result and demonstrates the effectiveness of the Bisection Method in locating the root of a nonlinear equation.

## 3.1 Computational Efficiency

The Bisection Method guarantees convergence provided a sign change exists within the selected interval. The interval width is reduced by half during each iteration, resulting in a logarithmic convergence rate. Although slower than methods such as Newton-Raphson, the Bisection Method is highly robust and reliable.

# 4 FLOWCHART


Bisection Method Flowchart
START
1. Define Function
Specify the function $f(x)$ whose root is to be found.
2. Search Interval
Scan the domain to find an interval $[a, b]$ such that $f(a) \cdot f(b) < 0$ (sign change).
3. Sign Change Found?
Check if $f(a) \cdot f(b) < 0$.
No
No Root Found
No sign change in the search domain.
STOP
Yes
4. Compute Midpoint
$c = \frac{a+b}{2}$
5. Update Interval
• If $f(a) \cdot f(c) < 0$, set $b = c$
• Else, set $a = c$
6. Tolerance Met?
Check if $|b - a| \leq$ tolerance
No
Yes
7. Display Root
Root $\approx \frac{a+b}{2}$
Display the root and iteration count.
END

# 5 BISECTION METHOD CODE

```
% Define the nonlinear function
f = @(x) x.^3 - x - 2;

% Search domain for automatic interval detection
domain = -10:1:10;
found = false;

% Locate an interval containing a sign change
for i = 1:length(domain)

    if f(domain(i))*f(domain(i+1))<0

        a = domain(i);
        b = domain(i+1);

        found = true;

        break
    end
end

% Stop execution if no valid interval is found
if ~found
    error('No sign changed detected in the search interval.')
end

% Display initial interval
fprintf('a=%d',a)
fprintf('b=%d',b)

% Convergence tolerance and iteration counter
tol = 1e-6;
iter = 0;

fprintf('Iter\t a\t\t b\t\t c\t\t')

% Perform Bisection Method iterations
while abs(b-a) > tol

    % Compute midpoint
    c = (a+b)/2;

    % Select subinterval containing the root
    if f(a)*f(c) < 0
        b = c;
    else
        a = c;
    end

    % Update iteration count
    iter = iter + 1;

    % Display iteration results
    fprintf('%d\t %.6f\t %.6f\t %.6f\t \n', iter,a,b,c)

end
```

```
% Calculate final root approximation
root = (a+b)/2;

% Display final results
fprintf('Root = %.6f\n', root)
fprintf('Iterations = %d\n', iter)

% Generate data for function plot
x = 0:0.1:3;
figure

% Plot function curve
plot(x,f(x),LineWidth=2)
grid on
hold on

% Mark computed root on graph
plot(root,f(root),'ro',MarkerSize=10)

% Add graph labels and title
xlabel('x')
ylabel('f(x)')
title('Bisection Method Root')
```

# 6 OUTPUT

## 6.1 INITIAL VALUES

| Parameter | Value |
|---|---|
| Lower Bound (a) | 1 |
| Upper Bound (b) | 2 |

## 6.2 CONVERGENCE TABLE

***Table 1*** *Convergence history of the Bisection Method for f(x)*

| Iteration | a | b | c = (a+b)/2 |
|---|---|---|---|
| 1 | 1.500000 | 2.000000 | 1.500000 |
| 2 | 1.500000 | 1.750000 | 1.750000 |
| 3 | 1.500000 | 1.625000 | 1.625000 |
| 4 | 1.500000 | 1.562500 | 1.562500 |
| 5 | 1.500000 | 1.531250 | 1.531250 |
| 6 | 1.515625 | 1.531250 | 1.515625 |
| 7 | 1.515625 | 1.523438 | 1.523438 |
| 8 | 1.519531 | 1.523438 | 1.519531 |
| 9 | 1.519531 | 1.521484 | 1.521484 |
| 10 | 1.520508 | 1.521484 | 1.520508 |

| Iteration | a | b | c = (a+b)/2 |
|---|---|---|---|
| 11 | 1.520996 | 1.521484 | 1.520996 |
| 12 | 1.521240 | 1.521484 | 1.521240 |
| 13 | 1.521362 | 1.521484 | 1.521362 |
| 14 | 1.521362 | 1.521423 | 1.521423 |
| 15 | 1.521362 | 1.521393 | 1.521393 |
| 16 | 1.521378 | 1.521393 | 1.521378 |
| 17 | 1.521378 | 1.521385 | 1.521385 |
| 18 | 1.521378 | 1.521381 | 1.521381 |
| 19 | 1.521379 | 1.521381 | 1.521379 |
| 20 | 1.521379 | 1.521380 | 1.521380 |

## 6.3 GRAPH

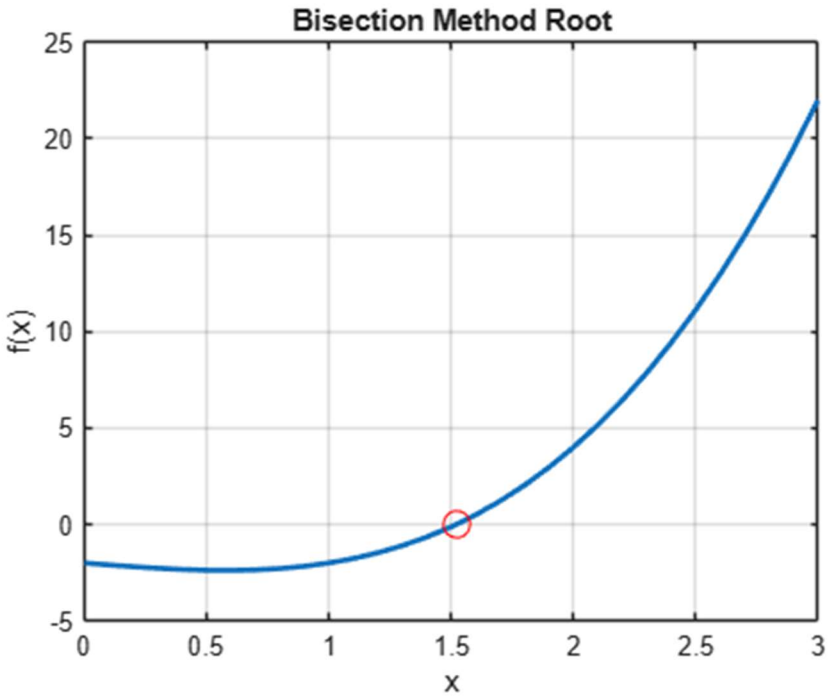



***Figure 1*** *Graph of f(x) showing the root obtained using the Bisection Method.*

# 7 RESULTS

The MATLAB implementation successfully converged to the root of the nonlinear equation:

$$f(x) = x^3 - x - 2$$

Starting from the automatically detected interval [1,2], the algorithm required 20 iterations to satisfy the specified tolerance of $1 \times 10^{-6}$. The computed root was found to be:

$$x = 1.521380$$

The convergence table demonstrates the progressive reduction of the search interval, while the graphical representation confirms that the computed value corresponds to the point where the function crosses the x-axis.

| **Quantity** | **Value** |
|---|---|
| Function | $x^3 - x - 2$ |
| Initial Interval | [1,2] |
| Root | 1.521380 |
| Tolerance | $1 \times 10^{-6}$ |
| Iterations | 20 |

# 8 CONCLUSION

A MATLAB-based Bisection Method solver was successfully developed and tested. The algorithm accurately located the root of the nonlinear equation $x^3 - x - 2 = 0$ and demonstrated reliable convergence. The project strengthened understanding of numerical root-finding techniques, iterative algorithms, and MATLAB programming.

Future work may include extending the solver to other root-finding techniques such as the False Position, Secant, and Newton-Raphson methods for performance comparison.